ملاحظات الدرس — لك

الطرق التقريبية والموازنات

الحلول المثلى، المستوى ٤، الدرس ٤

المدة
٢٥ دقيقة

المواد
هاتفك، أو حاسوب مع Colab، خريطة محلية فيها المسافات بين خمسة إلى عشرة أماكن

أهداف التعلّم

- تشرح الصعوبة الحسابية بكلمات بسيطة، ولماذا تُستعمل الطرق التقريبية.
- تنفّذ 2-opt وتقارن الطرق الفوقية للخروج من الحلول المحلية.
- تقيس فجوة الأمثلية، وتضع حدود العدالة علنًا.

النشاط: سباق المسارات

١. البناء (٦ دقائق): حدّد على الورق ٧ أماكن ونقطة بداية مع المسافات. وابنِ مسارًا بالذهاب دائمًا إلى أقرب مكان لم تزره، واجمع طوله.

٢. التحسين (٨ دقائق): حيثما تقاطع جزءان من المسار، اعكس الجزء الواقع بينهما (حركة 2-opt) وأعِد الحساب. وتوقّف حين لا تُقصّر أي حركة المسار.

٣. التحقّق (٦ دقائق): شغّل برنامج الدرس أو بحثًا شاملًا لإيجاد أفضل مسار حقيقي، واحسب فجوتك: (مسارك - الأفضل) ÷ مسارك.

٤. العدالة (٥ دقائق): اختر حدًّا لأطول رحلة من البداية إلى أي مكان. هل يلتزم مسارك به؟ وكم يكلّف الالتزام به من المسافة الكلية؟

استعمل خريطة محلية حقيقية إن استطعت، وأوقات الطريق تصلح كالمسافات. اعملوا في أزواج وتبادلوا المسارات ليتحقّق كلٌّ منكم من حسابات الآخر.

للنقاش

- لماذا تقبل الشركات مسارات لم يُثبت أحد أنها مثلى؟
- أيّ طريقة فوقية ستجرّب أولًا لجدول دراسي، ولماذا؟
- مَن ينبغي أن يقرّر أطول رحلة مقبولة في حافلة مدرسية؟

تحقّق من نفسك

- طول مسارك ٥٢ كم، ويقول حدّ أدنى ٥٠ كم. اكتب الفجوة، وجملة واحدة يفهمها مدير.

جرّبها هذا الأسبوع

اكتب تحسين 2-opt لمسار من خمس محطات بلغة بايثون، وتحقّق منه بمقارنته بكل المسارات الممكنة. ثم أضف حدًّا لأطول رحلة لأي راكب، وأعِد الحل، واكتب جملة واحدة عن كلفة هذا الحد ومَن يحميه.

ورقة عمل

الطرق التقريبية والموازنات

الحلول المثلى، المستوى ٤، الدرس ٤

الاسم ____ الفصل ____ التاريخ ____

١ **تنمذج حافلة الصباح لخمس قرى. الخطة A أقصر بكيلومترين في مجموعها، لكن أطفال إحدى القرى يركبون ٧٠ دقيقة، وفي الخطة B، الأطول بكيلومترين، لا يركب أي طفل أكثر من ٤٠ دقيقة. أيّهما يختار النموذج؟**

○ يتوقّف ذلك على الأهداف المتفق عليها: اعرض الخطتين وقرّر مع المتأثرين ○ A: الأقصر هو الهدف ○ B دائمًا

٢ **أكمل البرنامج ليطبع فجوة الأمثلية بالنسبة المئوية لمسار طوله ١٠٥ كم مقابل حدّ أدنى قدره ١٠٠ كم.**

```
found = 105   # km, the route the heuristic found
bound = 100   # km, no route can be shorter
gap = (found -  _________ ) / _________
print(round( _________ * gap, 1), "%")
```

بنك الكلمات: `100` `bound` `found` `10` `0`

٣ **يجد 2-opt دائمًا أقصر مسار إذا تركته يعمل حتى لا تفيد أي حركة.**

○ صح ○ خطأ

٤ **خمس محطات، والمسافات في d، والمسار الأول [0, 2, 1, 4, 3] (طوله ٣٥). أكمل حركة 2-opt: لكل i < j، اعكس الجزء r[i..j]، وأبقِ المسار الجديد إن كان أقصر.**

```
d = [[0, 3, 7, 6, 4], [3, 0, 4, 7, 6], [7, 4, 0, 3, 5],
     [6, 7, 3, 0, 2], [4, 6, 5, 2, 0]]
def length(r):
    return sum(d[r[i - 1]][r[i]] for i in range(len(r)))
r = [0, 2, 1, 4, 3]
for rounds in range(5):
    for i in range(1, 4):
        for j in range(i + 1, 5):
            pass  # reverse r[i..j]; keep it if shorter
print(r, length(r))
```

اكتب برنامجك:

٥ **صِل كل طريقة فوقية بفكرتها.**

التلدين المحاكى	يمنع التراجع عن الحركات الأخيرة مدة من الزمن
البحث المحرّم	تشغّل البحث المحلي من بدايات عشوائية كثيرة
الخوارزمية الجينية	يقبل بعض الحركات الأسوأ، وأقل كلما برد
البدء المتعدد	تجمع أجزاء من خطط جيدة

✎ **أين تقبل إجابة جيدة بما يكفي مع فجوة مقيسة، وأين تُصرّ على إثبات الأفضل؟ وما الذي يحسم ذلك؟**

الطرق التقريبية والموازنات

١ **يتوقّف ذلك على الأهداف المتفق عليها: اعرض الخطتين وقرّر مع المتأثرين**

لا تتفوّق أيّ خطة على الأخرى في الهدفين معًا، فالاختيار حكم على ما هو مهم: كيلومتران من الوقود ووقت القيادة مقابل ٣٠ دقيقة لأطفال قرية واحدة.

٢

```
found = 105   # km, the route the heuristic found
bound = 100   # km, no route can be shorter
gap = (found - bound) / found
print(round(100 * gap, 1), "%")
```

الفجوة = (found − bound) ÷ found = ٥ ÷ ١٠٥، وتُطبع 4.8 %.

٣ **خطأ**

إنه يتوقّف عند حل أمثل محلي: لا تفيد أي حركة 2-opt واحدة، لكن مسارًا أفضل قد يحتاج إلى نوع آخر من الحركات.

٤

```
d = [[0, 3, 7, 6, 4], [3, 0, 4, 7, 6], [7, 4, 0, 3, 5],
     [6, 7, 3, 0, 2], [4, 6, 5, 2, 0]]
def length(r):
    return sum(d[r[i - 1]][r[i]] for i in range(len(r)))
r = [0, 2, 1, 4, 3]
for rounds in range(5):
    for i in range(1, 4):
        for j in range(i + 1, 5):
            new = r[:i] + r[i:j + 1][::-1] + r[j + 1:]
            if length(new) < length(r):
                r = new
print(r, length(r))
```

عكس الجزء r[i..j] يبدّل ضلعين بضلعين آخرين.

٥

التلدين المحاكى — يقبل بعض الحركات الأسوأ، وأقل كلما برد

البحث المحرّم — يمنع التراجع عن الحركات الأخيرة مدة من الزمن

الخوارزمية الجينية — تجمع أجزاء من خطط جيدة

البدء المتعدد — تشغّل البحث المحلي من بدايات عشوائية كثيرة

تخرج الأربع من الحلول المثلى المحلية بطرق مختلفة: بقبول حركات سيئة، أو بالذاكرة، أو بإعادة التركيب، أو بالمحاولة من جديد في مكان آخر.